

DA3408 Assignment 3

Sujal Jayant Kumar (Roll No.: DA24B026)

Repository: https://github.com/SujalKumar06/assign3_da3408

1 Setup

I implemented the same preprocessing pipeline in Apache Spark (`spark_clean.py`) and Ray Data (`ray_clean.py`) and benchmarked both on the NYC TLC Yellow Taxi data: 38 monthly Parquet files (Jan 2022 to Feb 2025, 2.0 GB, **126,188,813 rows**) and the 265-row zone lookup table.

Both clusters run from one shared Docker image (Python 3.13, PySpark 4.2.0, Ray 2.58.0) with Docker Compose. Each cluster has one coordinator (the Spark master, or the Ray head started with 0 CPUs) and **2 workers** $\times$ **6 cores** $\times$ **8 GB**, so all data processing happens on two identical workers.

2 Pipeline and Parity

- Both pipelines read every rule, column name and type from one shared config, and call the same Python UDF.
- Steps: unify three different raw schemas, drop nulls and invalid trips, remove exact duplicates, join the zone table twice (pickup and dropoff) as a **shuffle join**, compute `avg_speed_mph` with a Python UDF, and write Parquet.
- `verify_parity.py` compares schemas, row counts per stage, and the full outputs with `exceptAll` in both directions.

Stage	Rows (Spark)	Rows (Ray)
raw	126,188,813	126,188,813
drop_nulls	118,072,836	118,072,836
validity_filters	111,083,668	111,083,668
dedup	111,083,664	111,083,664
join	111,083,664	111,083,664
speed_filter (output)	111,016,594	111,016,594
Rows only in Spark / only in Ray	0 / 0	(PARITY PASSED)

Table 1: Row counts per stage on the full dataset. The final outputs are identical row for row.

3 Cluster Orchestration

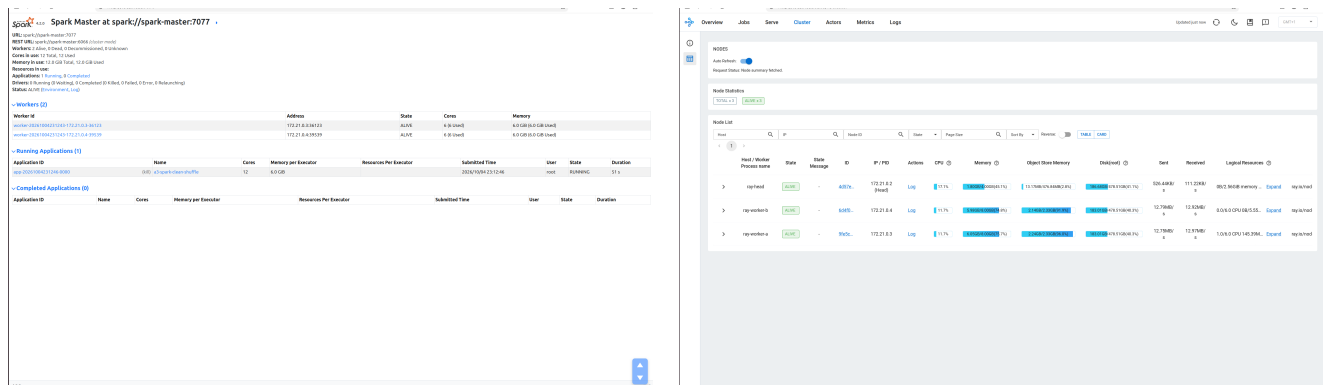


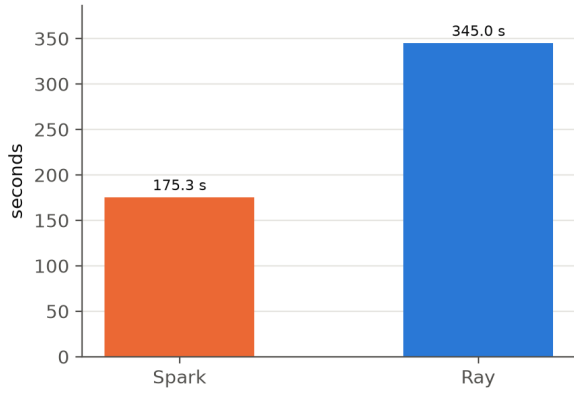
Figure 1: Both clusters during the full-data benchmark.

4 Performance Analysis

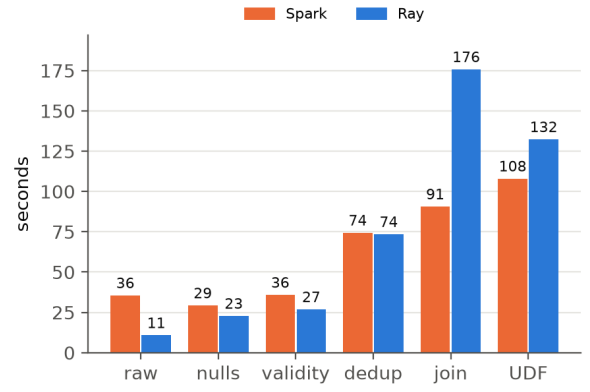
I ran each framework in two modes: a **timed** run (the whole pipeline end to end, ingestion to export) and a **stage-counts** run (each stage materialised and timed on its own). A monitor sampled CPU and memory of all cluster containers every 1 to 2s with `docker stats`, next to `top` snapshots.

Framework	Wall time (s)	Peak CPU %	Mean CPU %	Peak mem (GiB)	Mean mem (GiB)
Spark	175.3	1294	1057	11.95	10.70
Ray	345.0	1198	353	17.15	14.09

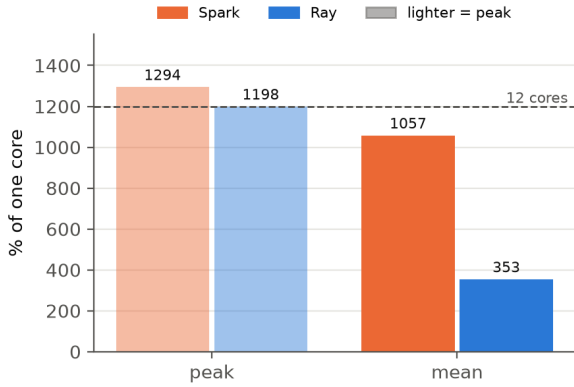
Table 2: Timed runs on the full data. CPU % is relative to one core (1200 % = all 12 worker cores).



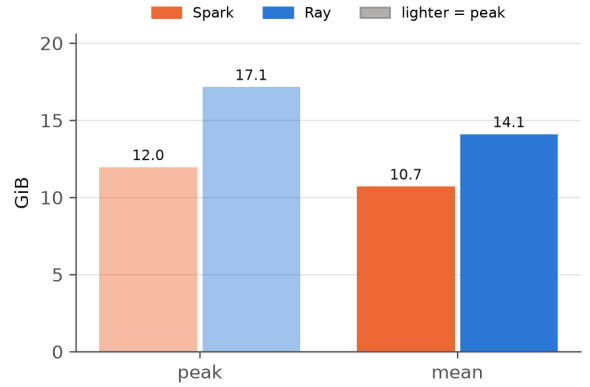
(a) End-to-end time.



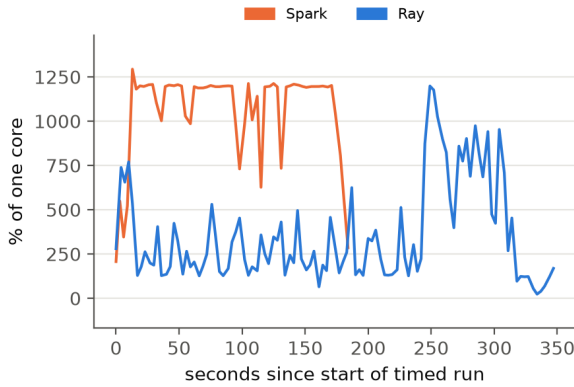
(b) Time per stage (stage-counts runs).



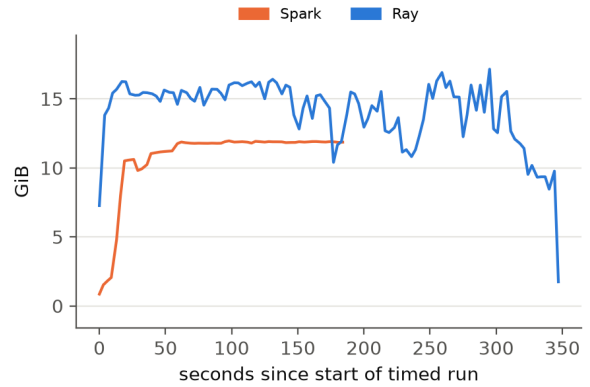
(c) CPU, peak and mean.



(d) Memory, peak and mean.



(e) CPU over the timed run.



(f) Memory over the timed run.

Figure 2: Benchmark results. Panels (c) to (f) are from the timed runs.

What I found:

- **Spark is $1.97\times$ faster end to end** (175.3s vs. 345.0s) and uses 30% less peak memory.
- **Ray is faster on row-parallel stages:** reading is $3.3\times$ faster and filtering $1.3\times$ faster, since these are independent Arrow batch operations.
- **Spark is faster on shuffle-heavy stages:** the join takes 91s in Spark and 176s in Ray. Under the same 8 GB per worker, Spark's shuffle spilled to disk, while Ray's shuffle tasks ran out of memory and were killed and retried (64 task retries in the timed run).
- **Spark keeps the cores busy:** mean CPU is 1057% for Spark and 353% for Ray. Figure 2(e) shows Ray idling for most of the run while it waits on the shuffle, object-store spilling and retries.

```
(base) sujals@madhav-LEGION-511n-5-16AHP9:~$ top
top - 05:00:26 up 2:02, 1 user, load average: 16.85, 13.63, 10.81
Tasks: 474 total, 2 running, 471 sleeping, 0 stopped, 1 zombie
%Cpu(s): 26.0 us, 3.6 sy, 0.1 ni, 69.9 id, 0.1 wa, 0.0 hi, 0.2 si, 0.0 st
MLB Mem : 31386.9 total, 1654.7 free, 19260.9 used, 10674.5 buff/cache
MLB Swap: 8192.0 total, 8191.4 free, 0.6 used, 12126.0 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 68168 root        20   0   13.8g  4.9g  26568 S 389.3 15.9 14:28.26 java
 68167 root        20   0   13.8g  4.8g  26428 S 60.1 15.7 14:28.75 java
 47787 root        20   0     0     0     0 R 22.6  0.0  0:39.19 kworker/u64:8+pm
 69758 root        20   0  57372 44148  5396 S 18.9  0.1  0:30.46 python3
11621 sujals     20   0 1449.9g 130352 89816 S 17.3  0.4  0:23.23 code
 69762 root        20   0  57372 44148  5396 S 16.6  0.1  0:30.36 python3
 69769 root        20   0  57372 44148  5396 S 11.0  0.1  0:31.25 python3
 47789 root        20   0     0     0     0 I  2.7  0.0  0:23.64 kworker/u64:11-pm
 69766 root        20   0  57372 44208  5456 S  2.0  0.1  0:29.59 python3
11584 sujals     20   0  48.4g 192644 15756 S  1.7  0.6  0:18.32 code
72142 sujals     39  19 475560 31652 25100 S  1.7  0.1  0:00.05 tracker-extract
 6341 sujals     20   0  25.6g 270940 167568 S  1.3  0.8  3:41.16 Xorg
11651 sujals     20   0 1449.9g 294924 100540 S  1.3  0.9  0:21.25 code
 6596 sujals     20   0 5388764 459468 173660 S  1.0  1.4  3:52.05 gnome-shell
 7054 sujals     39  19 883976 35732 18516 S  1.0  0.1  0:01.22 tracker-miner-f
11569 sujals     20   0 1452.2g 491348 157916 S  1.0  1.5  2:44.85 code
1741 root        20   0 2984132 74728 37916 S  0.7  0.2  0:26.88 containerd
2378 root        20   0 4142916 106992 59516 S  0.7  0.3  0:51.91 dockerd
 8595 sujals     20   0 898436 87024 45176 S  0.7  0.3  0:48.90 gnome-terminal-
 8833 sujals     20   0 12.1g 934952 368720 S  0.7  2.9  6:32.66 firefox
```

(a) Spark: Java executors busy.

```
Tasks: 506 total, 2 running, 503 sleeping, 0 stopped, 1 zombie
%Cpu(s): 3.3 us, 2.3 sy, 8.4 ni, 85.9 id, 0.1 wa, 0.0 hi, 0.0 si, 0.0 st
MLB Mem : 31386.9 total, 6225.0 free, 22177.6 used, 7705.5 buff/cache
MLB Swap: 8192.0 total, 4862.9 free, 3329.1 used, 9209.3 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
345171 root        35  15 8147444  2.4g  1.1g S 99.7  7.9  0:03.55 ray::HashShuffl
 6596 sujals     20   0 8386720 230220 48672 S 13.6  0.7  4:26.93 gnome-shell
248816 root        20   0 6914868 256192 100856 S 12.3  0.8  0:23.86 python
 74869 root        35  15 5372200 253836 88388 S  7.3  0.8  0:09.41 ray::_StatsActo
 6341 sujals     20   0  25.6g 160808 87028 S  6.0  0.5  4:21.39 Xorg
 8595 sujals     20   0 899104 60184 26404 S  4.0  0.2  0:52.57 gnome-terminal-
34314 sujals     20   0 2789828 227780 108368 S  4.0  0.7  0:29.73 Isolated Web Co
73200 root        20   0 3464696 211612 95520 S  3.3  0.7  0:07.43 ray-dashboard-D
249400 root        35  15 8677260  2.9g  2.1g S  3.3  9.4  0:08.69 ray::IDLE
249401 root        35  15 8677096  2.7g  2.0g S  3.3  8.7  0:07.97 ray::IDLE
73205 root        20   0 2397628 208916 93200 S  3.0  0.7  0:07.04 ray-dashboard-S
73809 root        20   0 458924 41472 37596 S  3.0  0.1  0:10.67 python3.13
249396 root        35  15 8742644  2.7g  2.0g S  3.0  8.8  0:07.46 ray::IDLE
249403 root        35  15 8677196  2.6g  1.9g S  3.0  8.5  0:07.29 ray::IDLE
249416 root        35  15 8677208  2.7g  1.7g S  3.0  8.8  0:06.88 ray::IDLE
73685 root        20   0 4867772 221396 50056 S  2.6  0.7  0:06.25 ray::DashboardA
74806 root        35  15 5454432 177420 87532 S  2.6  0.6  0:07.69 ray::_Autoscali
249397 root        35  15 8742756  2.5g  1.9g S  2.6  8.2  0:07.95 ray::IDLE
384236 root        35  15 8389852  3.8g  2.4g S  2.6 12.5  0:20.02 ray::IDLE
333382 root        35  15 7333344 723300 498608 S  2.6  2.3  0:01.58 ray::IDLE
73810 root        20   0 458924 41532 37876 S  2.3  0.1  0:08.42 python3.13
249402 root        35  15 8677032  2.8g  2.1g S  2.3  9.1  0:07.88 ray::IDLE
249415 root        35  15 8677000  2.9g  2.1g R  2.3  9.4  0:07.78 ray::IDLE
73097 root        20   0 795952 79044 49444 S  1.3  0.2  0:06.69 python3.13
 87368 root        35  15 5734612  2.4g  2.4g S  1.3  7.8  0:07.96 ray::IDLE_Spill
 88201 root        35  15 5734612 736764 781236 S  1.3  2.3  0:04.22 ray::IDLE_Spill
 88330 root        35  15 5734612  2.4g  2.4g S  1.3  7.8  0:06.45 ray::IDLE_Spill
121400 root        35  15 5734612  2.4g  2.3g S  1.3  7.7  0:07.34 ray::IDLE_Resto
121981 root        35  15 5734608  2.3g  2.3g S  1.3  7.5  0:06.60 ray::IDLE_Resto
122072 root        35  15 5734612  2.4g  2.4g S  1.3  7.8  0:07.56 ray::IDLE_Resto
122621 root        35  15 5734608  2.4g  2.4g S  1.3  7.8  0:08.74 ray::IDLE_Resto
122623 root        35  15 5734616  2.3g  2.3g S  1.3  7.6  0:06.65 ray::IDLE_Resto
```

(b) Ray: one shuffle task busy, many idle workers.

Figure 3: `top` during the timed runs.

5 UDF Deep-Dive: JVM vs. Python-Native

Framework	UDF stage (s)	How rows reach the Python function
Spark	108.0	JVM, pickled to a separate Python worker over a socket, results sent back
Ray	132.4	Arrow batch in the same Python process, converted to Python lists

Table 3: The `avg_speed_mph` UDF on 111 M rows, timed in isolation in the stage-counts runs.

What I found:

- In Spark, the UDF is the single most expensive stage, more than the shuffle join. Every row crosses the JVM to Python boundary twice, and the Python workers need memory next to the JVM heap. I had to lower the executor heap because the Python workers pushed the 8 GB workers to their limit.
- Ray avoids that boundary, yet its UDF stage was still slower here. Being Python-native removes the *transfer* cost, not the *per-row interpreter* cost: both engines still make 111 M Python calls.
- Ray also ran this stage under memory pressure. The stage-counts run keeps each stage’s output in the object store, and the UDF reads the widest data (the joined 111 M rows, about 30 columns, six of them text). With 2.5 GB of object store per worker, most of it was on disk: each worker’s spill total doubled from about 34 to 67 GB around this stage. `to_pylist` also turns three Arrow columns into Python objects about 4× larger, in 6 tasks per worker. Part of Ray’s 132 s is disk I/O, not UDF work.
- What I understood was Ray’s Python-native design pays off when the Python work per call is heavy or

vectorised. For a scalar row-level UDF, the interpreter dominates, and vectorising (Spark `pandas_udf`, Ray NumPy batches) matters more than the framework.

6 Issues I Hit

- **Spark:** planning ran out of memory (fixed by disabling constraint propagation); executors got a 1 GB heap until `-master` was passed to `spark-submit`.
- **Ray:** the actor-based shuffle deadlocked (switched to `HASH_SHUFFLE_V2`), a column-less empty block crashed it (fused the cleansing maps), and blocks are capped at 32 MB to fit worker memory.

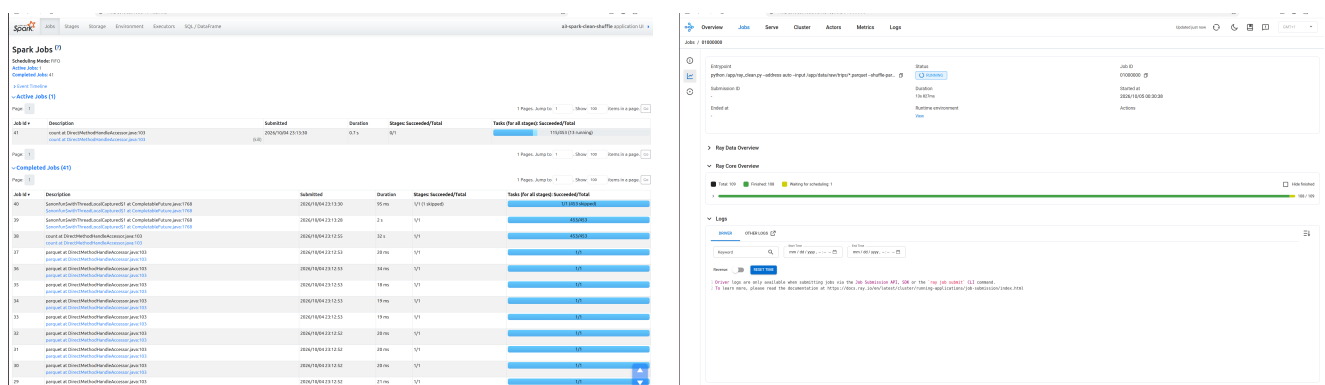
7 Conclusion

Winner for this workload: Spark. For a cleaning, deduplication and join pipeline over 126 M rows, Spark was 2× faster, used less memory and kept the cores busy. Ray was faster at ingestion and row-parallel filtering but lost time in its shuffles.

BI-first project: Spark. Joins, aggregations and SQL-style reporting benefit from Spark’s optimiser and its spilling shuffle, which is exactly the profile of this benchmark.

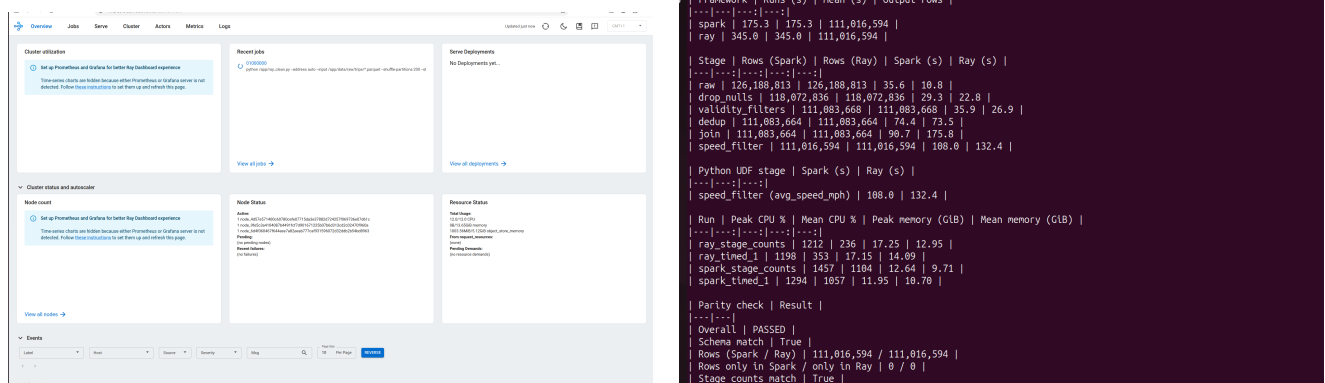
AI-first project: Ray. When the pipeline feeds training or inference with arbitrary Python (tokenisers, image decoding, model calls, GPUs), Ray's Python-native workers and its path into Ray Train and Ray Serve outweigh its weaker shuffles. In practice I would use Spark for the heavy relational ETL and Ray Data for the Python-heavy, ML-facing last mile.

A Additional Evidence



(a) Spark application UI during the full-data run.

(b) Ray dashboard job view.



(c) Ray overview: 3 nodes, 12.0/12.0 CPU in use.

(d) Benchmark output: parity passed and summary tables.

Figure 4: Jobs on both clusters and the benchmark output.